Fail-Safe In-Orbit Reprogramming of a Spacecraft Controller

Software Architecture

Reference	TBD **[TBD-1]**
Issue / revision	Issue 1, Revision 0 — DRAFT
Date	11 October 2026
Author	Andrew Toma
Open items	3 TBD, 8 TBR (App. A)

Document change record

Issue	Rev.	Date	Change
1	0	2026-10-11	First draft

1.0 Purpose and Scope

This document records the software architecture of the System: its components, what each does, how they connect, and the design decisions behind them [1, Sec. 4.2.3, p. 33]. It covers the three software items of the requirements [6, Sec. 3.1].

The architecture concentrates on "the most pressing risks" [3, Sec. 4]: the eight catastrophic and critical hazards that do not yet meet the fault tolerance rule [5, Sec. 5.0]. The software qualities it is built for are the two a high-risk project emphasises, fault tolerance and security [3, Sec. 4], and the testability the requirements demand.

It names no hardware. It says what each component does, not which device does it, and ends with what the hardware must provide. The detailed design of each component, with its algorithms and data, is recorded before that component is coded [1, Sec. 4.3.2, p. 34].

How to read it. Section 3.0 shows every component and one update on two figures. Section 5.0 holds the design decisions, each with the options weighed and the hazards it targets. Section 6.0 gives each component's role and the requirements it meets. Section 8.0 lists what the hardware must provide.

2.0 Documents

[1] NASA, *NASA Software Engineering Requirements*, NPR 7150.2D.
[2] NASA, *NASA Software Engineering Handbook*, NASA-HDBK-2203, Version D, Topic 5.13 "SwDD – Software Design Description".
[3] NASA, *NASA Software Engineering Handbook*, NASA-HDBK-2203, Version D, SWE-057 "Software Architecture".
[4] *Fail-Safe In-Orbit Reprogramming of a Spacecraft Controller — Concept of Operations*, Issue 1, Revision 1, Draft.
[5] *Fail-Safe In-Orbit Reprogramming of a Spacecraft Controller — Hazard Analysis*, Issue 1, Revision 0, Draft.
[6] *Fail-Safe In-Orbit Reprogramming of a Spacecraft Controller — Software Requirements Specification*, Issue 1, Revision 0, Draft.
[7] *Fail-Safe In-Orbit Reprogramming of a Spacecraft Controller — Software Classification*, Issue 1, Revision 1, Draft.
[8] NASA, *NASA Software Safety Guidebook*, NASA-GB-8719.13, 2004.

[9] MCUboot project, *Bootloader* design document, `docs/design.md`, commit 1d2643c, 11 September 2026.
[10] G. Reeves and T. Neilson, "The Mars Rover Spirit FLASH Anomaly", IEEE Aerospace Conference, paper 1426, 2005.
[11] NASA, *Space System Protection Standard*, NASA-STD-1006A, 15 July 2022.
[12] NASA, *Fault Management Handbook*, NASA-HDBK-1002, Draft 2, 2 April 2012. A draft, not a released standard.

3.0 System Overview

The System has three software items [6, Sec. 3.1], each made of components (Figure 1). The ground software turns the operator's decisions into signed commands and firmware, and keeps every report that comes back. The onboard computer software checks everything that arrives and passes it on; between contacts it stores the controller's reports, watches the controller, and restores it when no firmware can run. The controller firmware acts on commands, reports its state and handles its own faults. It tries new firmware before keeping it and returns to working firmware on its own when the new firmware fails: a start-up selector chooses which copy runs, and the last-resort firmware takes over when the main firmware cannot. Figure 2 follows one update from release to confirmed firmware.

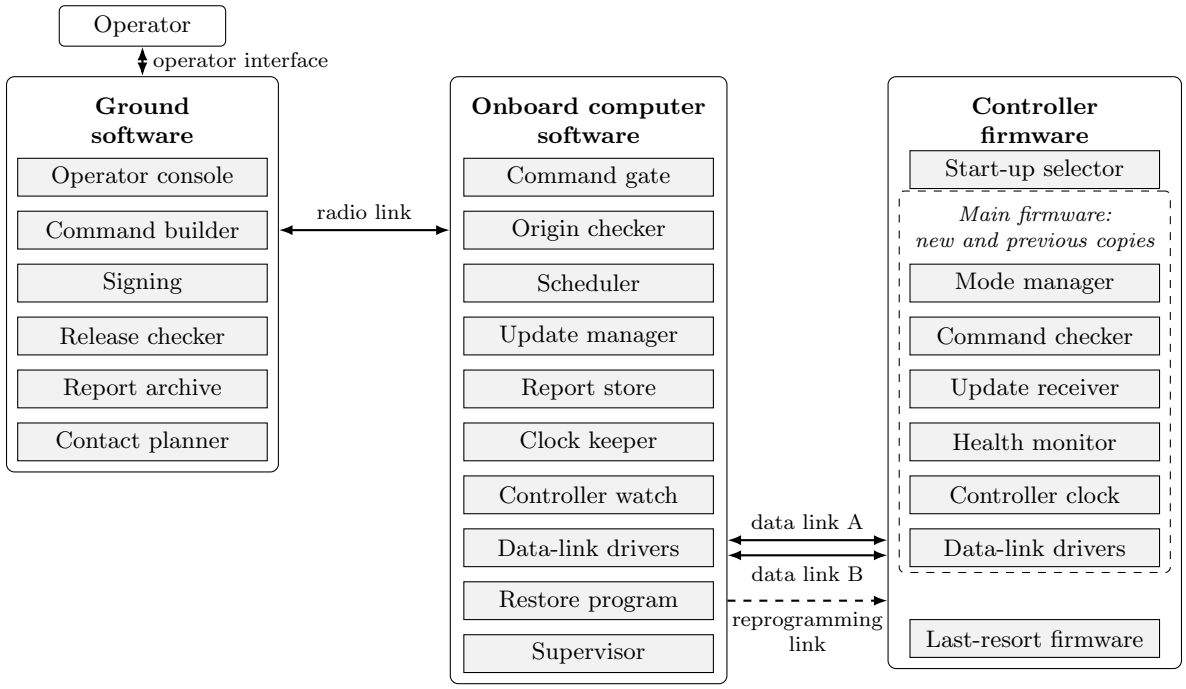


Figure 1: The three software items and their components. The data link has two connections, A and B. The dashed arrow is the reprogramming link, which reaches the controller whatever its memory holds.

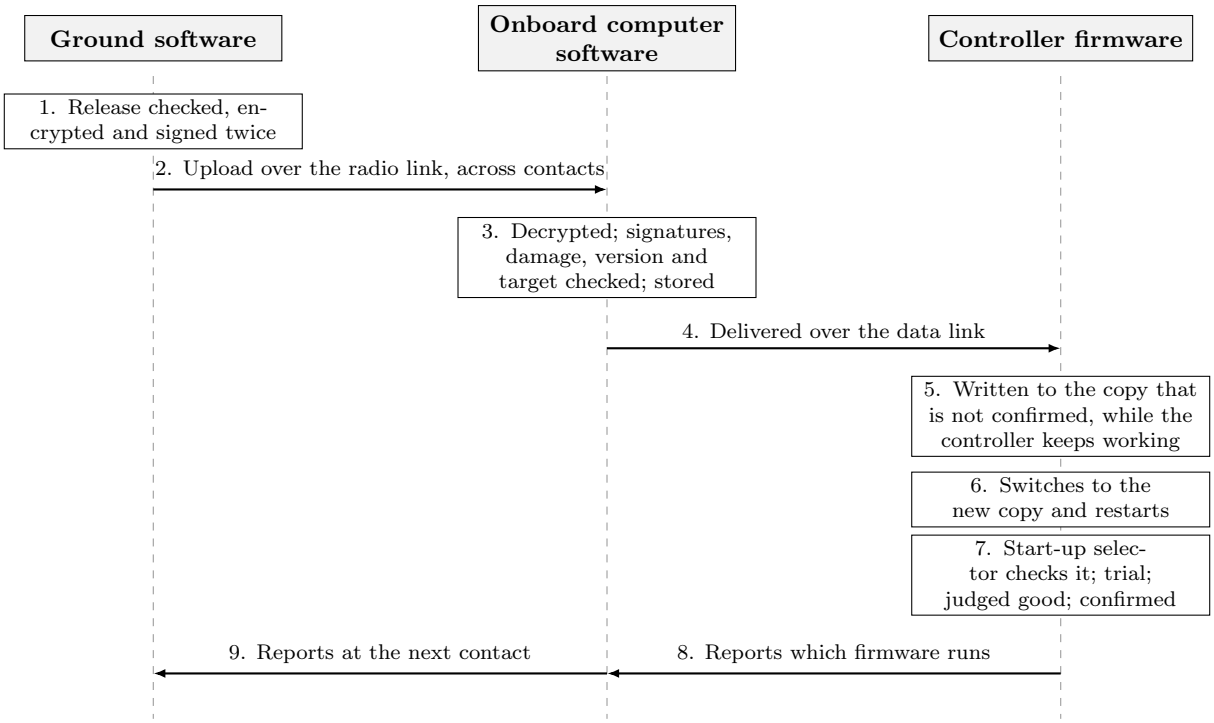


Figure 2: One update, from release to confirmed firmware. If the trial fails, the controller returns to the previous copy (Figure 3).

4.0 Constraints

Three constraints of the ConOps change the design [4, Secs. 1.1.2 and 8.0].

C-3 One engineer builds the System. Independence between components comes from separate code, not from separate teams.

R-4 There is one onboard computer. Its permanent loss stops commands, reports and restores. The risk is accepted, so H-4 cannot meet the fault tolerance rule [5, Sec. 5.2].

C-4 The System runs on two given development boards. The design does not depend on them; they are checked against it at the end of this document.

5.0 Design Decisions

Each decision targets hazards that do not yet meet the fault tolerance rule [5, Sec. 5.2]. It states the problem, weighs the options, and lists the requirements the choice adds; these enter the requirements at their next revision. Where a proven design exists, it is one of the options [3, Sec. 4].

Options are compared by the smallest number of failures that, together, cause the hazard with that option in place. The counts leave out the permanent loss of the onboard computer, an accepted risk (R-4). They are design targets, which the hazard analysis confirms in its design phase [5, Sec. 1.0]. Chosen: YES or NO.

5.1 Start-Up and Fall-Back

Problem. Every firmware copy, including the one the restore puts back, shares the start-up code and the data kept over restarts. One defect there stops them all, and the restore brings it back [5, Sec. 5.2]. Targets H-1 and H-8.

	Option	H-1	H-8	Chosen
a	The last-resort firmware runs first and chooses; the restore puts it back	2	1	NO
b	A start-up selector runs first and chooses between two copies; a new copy runs as a test and is reverted unless it confirms itself, as in MCUboot [9, Sec. Boot Swap Types]; the restore puts back the same selector	2	1	NO
c	As b, plus a last-resort firmware sharing no code with the selector or either copy; the restore puts it in the selector's place	3	2	YES

Choice. Option c **[TBR-1]**, shown in Figures 3 and 4. It keeps MCUboot's test and revert, which exists to keep bad firmware from leaving a device unusable [9, Sec. Boot Swap Types], and adds a last-resort firmware sharing no code with the rest [6, REQ-009]. The last-resort firmware is the controller's safe mode: it still accepts commands from the ground [12, Sec. 6.4.3.2.4, p. 85].

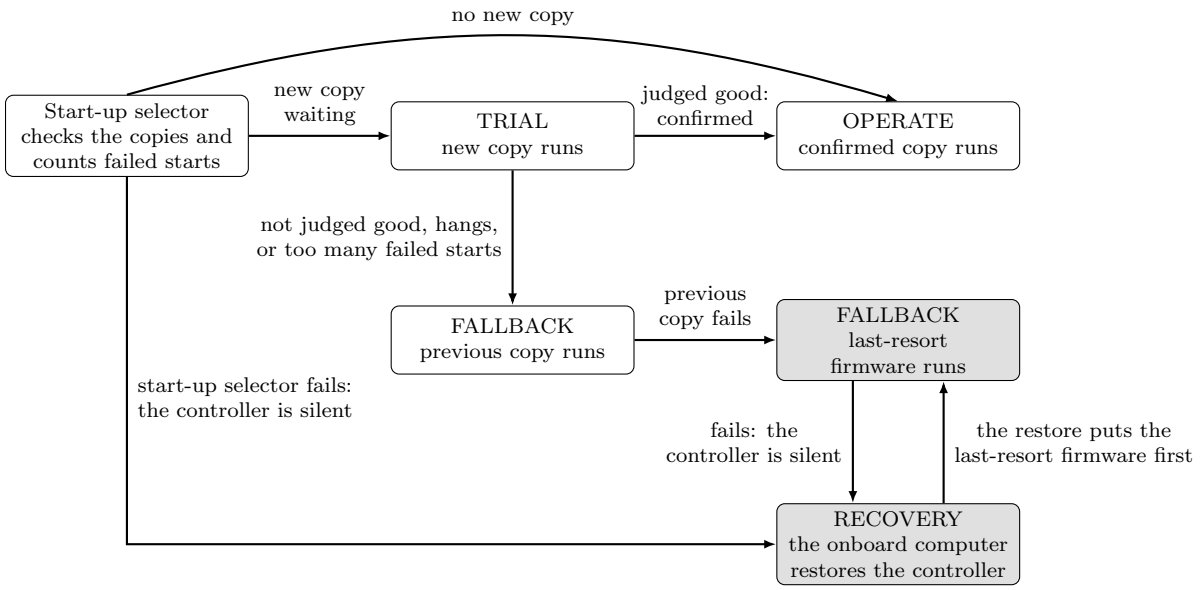


Figure 3: Start-up and fall-back. White: the two-copy test and revert of MCUboot [9, Sec. Boot Swap Types]. Grey: added for a controller that nobody can reach, the last-resort firmware and the restore.

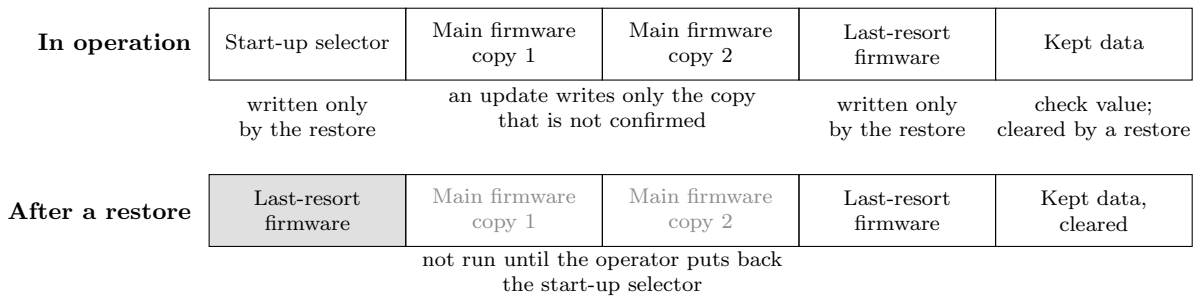


Figure 4: The controller's memory, not to scale. After a restore, the last-resort firmware (grey) sits in the start-up selector's place and runs first.

Adds.

- The start-up selector shall share no code with the main firmware or the last-resort firmware.
- The restore shall put the last-resort firmware in the start-up selector's place, so that it runs first.
- The onboard computer software shall put back the start-up selector and the main firmware through the reprogramming link when the operator orders it.

5.2 The Way Back In

Problem. A restore that goes through any controller software fails whenever that software fails, which the requirements forbid [6, REQ-056]. Targets H-1, H-3 and H-4.

	Option	H-1	Chosen
a	The restore is a command that the controller's firmware carries out	2	NO
b	The reprogramming link loads the controller's memory whatever it holds; the onboard computer drives it and can restart the controller	3	YES
c	Two such links, independent of each other	4	NO

Choice. Option b [**TBR-2**]. It follows the principle of hardware commands, which a spacecraft executes on receipt without any flight software [11, Sec. 3.2, p. 9]. Such a command let the Spirit rover's team regain control while its flight software kept resetting: it made the software start without its flash memory [10, pp. 4 and 9]. One link meets the rule; a second adds a connection to build and to check between uses [6, REQ-059] for a margin the rule does not ask for.

Adds.

- The reprogramming link shall load the controller's memory whatever that memory holds.
- The onboard computer software shall be able to restart the controller.
- No controller firmware shall be able to disable the reprogramming link.

5.3 Restore Apart From Update

Problem. The update path and the restore share onboard computer software and stored firmware. One fault there stops both until the outside system corrects it (A-6) [5, Sec. 5.2]. Targets H-3.

	Option	H-3	Chosen
a	One program updates and restores, from one firmware store	2	NO
b	A restore program, separate from the update manager, with its own firmware store	3	YES

Choice. Option b [**TBR-3**]. New firmware can still reach the controller through the restore program, which loads firmware from the ground [6, REQ-058], when the update path fails.

Adds.

- The restore program shall share no code or stored data with the update manager.

5.4 Proof of Origin

Problem. One means proves that both commands and firmware come from the operator. If it is disclosed, another party can command the controller and load firmware [5, Sec. 5.2]. Targets H-2 and H-5.

	Option	H-2	H-5	Chosen
a	One secret, held on the ground and on board, checks commands and firmware	1	1	NO
b	Signatures made with one key; on board is only the means to check them, as in MCUboot [9, Sec. Security]	1	1	NO
c	Encryption of every command and firmware image [11, Sec. 4.1.1, p. 9], plus one signature; the two keys kept in two separate places	2	2	NO
d	As c, with a second signature; the three keys kept in three separate places	3	3	YES

Choice. Option d **[TBR-4]**, shown in Figure 5. Each disclosure exposes one key, so another party needs three. The command stack must be encrypted [11, Sec. 4.1.1, p. 9], and encryption with authentication is the recommended defence in depth [11, Sec. 4.1.1.c, p. 10]. The controller checks firmware signatures again before running it [6, REQ-006], with code different from the onboard computer's, so that one defect in that code cannot pass forged firmware through both checks [8, Sec. 7.4.2, p. 141].

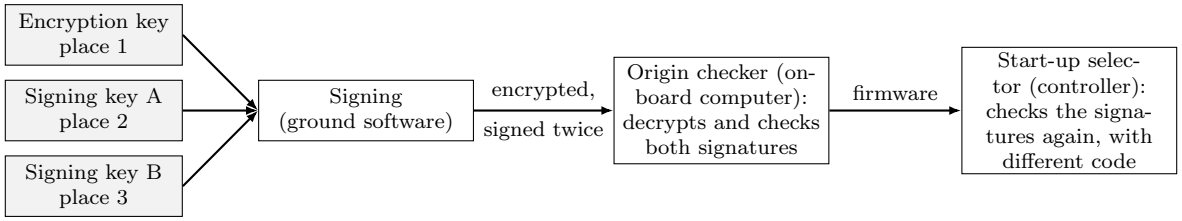


Figure 5: Proof of origin. Each key is kept in its own place, so one disclosed key is not enough. Commands stop at the onboard computer's check; firmware is checked again on the controller.

Adds.

- Every command and firmware image shall be encrypted from the ground software to the onboard computer software [11, Sec. 4.1.1, p. 9].
- Every command and firmware image shall carry two signatures, made with two keys kept apart from each other and from the encryption key.
- The onboard computer software shall act on a command or firmware image only when it decrypts and both signatures are valid.
- The controller firmware shall check firmware signatures with code different from the onboard computer software's.

5.5 Two Clocks

Problem. Expiry, arm time-outs, scheduled commands, time-tags and the trial time limit all depend on the onboard clock. A wrong clock alone runs a scheduled command at the wrong time and gives reports wrong times [5, Sec. 5.2]. Targets H-6 and H-9.

	Option	H-6	H-9	Chosen
a	The onboard clock alone, compared with ground time at each contact [6, REQ-072]	1	1	NO
b	Counts instead of times wherever possible, such as report and command numbers	1	1	NO
c	A second clock in the controller, with its own time source, set only from ground time; each processor makes every time check with its own clock, and every report carries both times	2	2	YES

Choice. Option c [**TBR-5**], shown in Figure 6. A wrong clock on one processor is caught by the other.

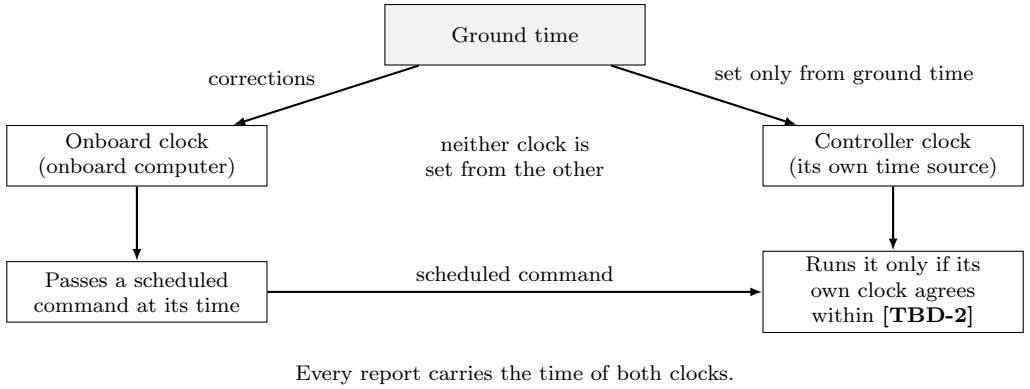


Figure 6: Two clocks. Neither is set from the other, so one wrong value cannot reach both.

Adds.

- The controller firmware shall keep its own clock, from a time source separate from the onboard computer's.
- The controller's clock shall be set only from ground time.
- The controller firmware shall refuse a command past its expiry, by its own clock.
- The controller firmware shall run a scheduled command only when its own clock agrees with the scheduled time within [**TBD-2**].
- Every report shall carry the time of both clocks.

5.6 Held Data

Problem. A bit error can change a held command, the arm state or the kept data after its check and before its use (A-5) [5, Sec. 5.2]. Targets H-6.

	Option	H-6	Chosen
a	Each item is checked once, when it arrives	1	NO
b	Each held item is stored with a check value and checked again just before use; the arm state is stored as a value that no single bit error turns into armed	2	YES
c	Memory that corrects bit errors on its own	2	NO

Choice. Option b [**TBR-6**]. A bit error must also produce a matching check value, a second failure. MCUboot notes the same risk: skipping the check before use lets an image changed after a good check run unchecked [9, Sec. Integrity Check]. Unlike option c, option b needs no hardware feature; option c can be added where the hardware has it.

Adds.

- Every held command and the kept data shall be stored with a check value and checked just before use.
- The arm state shall be stored so that no single bit error makes it armed.

5.7 Two Kinds of Connection

Problem. The two data-link connections share software at both ends, so one defect cuts both (R-1) [5, Sec. 5.2]. Targets H-4.

	Option	Failures to cut both	Chosen
a	Two connections of one kind, one driver at each end	1	NO
b	Two connections of one kind, a separate driver for each	1	NO
c	Two different kinds of connection, a separate driver for each	2	YES

Choice. Option c **[TBR-7]**. One defect, or one fault common to a kind of connection, now cuts one connection; the other carries on (O-3). H-4 still falls to the loss of the onboard computer (R-4).

Adds.

- The two data-link connections shall be of different kinds.

6.0 Components

Each component, its role, and the requirements it meets [2, Sec. 3.3]. Every requirement [6, Secs. 4.0–7.0] is met by at least one component; requirements that the decisions add are allocated when they enter the requirements.

All components are new code and safety-critical [7, Sec. 4.0], with four exceptions. The start-up selector, the origin checker and signing are built around a published cryptographic library. The contact planner is not safety-critical **[TBR-8]**: it passes plans to the command builder, which checks every command it builds.

The components from the mode manager to the data-link drivers form the main firmware; the controller keeps two copies of it. Figure 3 shows which copy runs in each controller mode [4, Sec. 3.4].

Component	Role	Requirements
Controller firmware		
Start-up selector	Runs first at every start. Sets a safe state, checks each firmware copy for damage and origin, counts failed starts, and starts the new, previous or last-resort firmware.	REQ-004 to 006, REQ-013 to 019, REQ-023, REQ-113
Mode manager	Holds the controller mode, runs the trial, and confirms new firmware or gives it up.	REQ-001 to 003, REQ-011 to 013, REQ-020, REQ-021, REQ-024 to 026, REQ-113
Command checker	Checks every command for mode, values, arming, expiry and repeats; checks held commands again before running them; answers and logs.	REQ-027 to 036, REQ-100
Update receiver	Stores new firmware in the free copy, never over the previous or last-resort firmware.	REQ-007, REQ-008
Health monitor	Tests the controller, detects faults, restarts it when it stops responding, and sends health and event reports.	REQ-022, REQ-037 to 041
Controller clock	Keeps the controller's own time, set only from ground time.	REQ-030
Data-link drivers	One driver for each data-link connection, sharing no code.	REQ-042, REQ-099
Last-resort firmware	A separate program sharing no code or kept data with the others. Sets a safe state, answers commands, sends reports, and receives new firmware.	REQ-004, REQ-009, REQ-010
Onboard computer software		
Command gate	Checks every command for origin, newness, expiry and arming, passes it on, and logs it.	REQ-064 to 067, REQ-087, REQ-100, REQ-112
Origin checker	Decrypts commands and firmware, checks their two signatures, and replaces the means of checking them on command.	REQ-043, REQ-064, REQ-084 to 086

Component	Role	Requirements
Scheduler	Holds scheduled commands and passes each on once, at its run time, across restarts.	REQ-068, REQ-069
Update manager	Checks new firmware, stores it, and delivers it over the data link, resuming after any interruption.	REQ-043 to 053, REQ-063
Report store	Checks, numbers and time-tags every report, stores reports between contacts, and sends them at the next one.	REQ-070, REQ-071, REQ-074 to 079
Clock keeper	Compares the onboard clock with ground time and applies corrections.	REQ-072, REQ-073
Controller watch	Notices a silent controller, restarts it, and calls the restore when no firmware runs.	REQ-061
Supervisor	Restarts any program of the onboard computer software that crashes.	REQ-062
Data-link drivers	One driver for each data-link connection, sharing no code.	REQ-080 to 082, REQ-099
Restore program	A separate program with its own firmware store. Restores the controller through the reprogramming link, loads firmware from the ground through it, and checks its store and the link between uses.	REQ-054 to 060, REQ-083
Ground software		
Operator console	Lets only the operator in, and shows reports, unanswered commands and the running firmware.	REQ-091, REQ-094, REQ-095
Command builder	Builds every command from the shared command list and checks it before it is sent.	REQ-089, REQ-100
Signing	Encrypts and signs every command and firmware image with three keys kept in separate places, and keeps a recoverable copy of the means of proving origin.	REQ-088, REQ-096
Release checker	Checks every firmware release before it is sent.	REQ-090
Report archive	Keeps every report received and checks it for damage.	REQ-092, REQ-093
Contact planner	Plans commands, uploads and report retrieval for coming contacts.	REQ-097
All software items		
Every component	Rules that apply to all software.	REQ-098, REQ-101 to 111

7.0 Interfaces

Each interface is specified byte by byte before any code is written. Interfaces between components of one software item are set in its detailed design.

Interface	Between	Carries
Operator interface	Operator and ground software	The operator's decisions; reports and the System's state
Radio link	Ground software and onboard computer software	Signed commands and firmware; answers and reports
Data link A	Onboard computer and controller	Commands, answers, reports and firmware
Data link B	Onboard computer and controller	The same as data link A, over a different kind of connection
Reprogramming link	Restore program and controller	The controller's restart, and its memory contents

8.0 Hardware Needs

The hardware must provide what the decisions above assume. Each need is checked against the two boards before this document is released **[TBD-3]**.

Need	Needed by
A loading path into the controller that works whatever its memory holds, driven by the onboard computer	Section 5.2
A way for the onboard computer to restart the controller	Section 5.2
No way for controller firmware to disable that loading path	Section 5.2
A way for the restore to make the last-resort firmware run first	Section 5.1
Room in the controller for the start-up selector, two main firmware copies, the last-resort firmware and the kept data	Section 5.1
Kept data in the controller that survives a restart and a power loss	REQ-017
Room on the onboard computer for two separate firmware stores	Section 5.3
Processing in the controller to check two signatures within the restart time	Section 5.4; REQ-023
A time source in the controller separate from the onboard computer's	Section 5.5
Two different kinds of connection between the onboard computer and the controller	Section 5.7

A Open Items

The values below are not yet known.

TBD-1 Project name and document reference.

TBD-2 Largest allowed difference between the controller's clock and the time of a scheduled command.

TBD-3 Check of each hardware need against the two boards.

The proposals below await the author's acceptance.

TBR-1 Start-up and fall-back: option c (Section 5.1).

TBR-2 The way back in: option b (Section 5.2).

TBR-3 Restore apart from update: option b (Section 5.3).

TBR-4 Proof of origin: option d (Section 5.4).

TBR-5 Two clocks: option c (Section 5.5).

TBR-6 Held data: option b (Section 5.6).

TBR-7 Two kinds of connection: option c (Section 5.7).

TBR-8 The contact planner is not safety-critical (Section 6.0).